applied sciences

MDPI

Article

Design of a Lightweight Edge-AI System for Predictive Maintenance on ESP32-S3

Gaurav Kumar *, Maris Terauds *, Amal Ajayakumar Raji, Janis Semenako, Vladimirs Smolaninovs, Pauls Eriks Sics and Arun Kumar Malayidinja Poikayil Thankappan

Institute of Photonics, Electronics and Telecommunications, Riga Technical University, LV-1002 Riga, Latvia; amal.ajayakumar-raji@rtu.lv (A.A.R.); janis.semenako@rtu.lv (J.S.); vladimirs.smolaninovs@rtu.lv (V.S.); pauls-eriks.sics@rtu.lv (P.E.S.); arun.malayidinja@edu.rtu.lv (A.K.M.P.T.)

* Correspondence: gaurav.kumar@edu.rtu.lv (G.K.); maris.terauds@rtu.lv (M.T.)

Abstract

While predictive maintenance increasingly relies on artificial intelligence, strict dependence on cloud computing introduces network latency and demands continuous connectivity, creating critical bottlenecks for time-sensitive industrial applications. To overcome this, we introduce a novel hybrid edge-cloud architecture, which allows deploying an ultra-low-power microcontroller (ESP32-S3) without dedicated AI acceleration hardware to perform complete, operational, predictive maintenance on ultra-constrained embedded hardware. The edge model is optimized to be very small to ensure that increasing model complexity does not cause inference latency to exceed 100 ms or make real-time operation infeasible. We created a very compact INT8-quantized neural network to perform the simultaneous classification of faults and estimation of Time-to-Failure (TTF) with a deterministic mean inference time of 42.3 ms. It dynamically estimates prediction confidence, processes high-confidence predictions locally, and offloads uncertain predictions to a higher-capacity cloud model, and recovers 97.3% of the cloud accuracy gain at 92% of the cloud latency budget. An asymmetric loss function penalizes over-prediction of the remaining useful life, and thus it provides conservative and safe warnings of fault. Operators' interpretability is improved with Shapley Additive exPlanations (SHAP) and natural-language recommendations. Network outages of up to 50% have not influenced the safety-critical fault recall (above 0.924), so graceful degradation is reached when the network is used in real time in industrial applications. The edge-first with adaptive cloud fallback approach is demonstrated to be technically feasible for a full predictive maintenance workflow—including inference, confidence fusion, and explainability on a low-cost commercial microcontroller.

Keywords: edge AI; predictive maintenance; TinyML; ESP32-S3; embedded neural networks; hybrid edge-cloud architecture; quantized inference; industrial IoT; real-time diagnostics; explainable AI

Check for updates

Academic Editor: Benoit Iung

Received: 19 March 2026
Revised: 12 May 2026
Accepted: 18 May 2026
Published: 25 May 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Industrial equipment such as motors, pumps, and turbines plays a critical role in manufacturing, transportation, and infrastructure sectors. Unexpected failures of these assets lead to significant downtime, safety hazards, and substantial economic losses. The typical reactive approach and rules-based maintenance regime lead to over-service or late detection of faults. The wide solution is predictive maintenance, which is based on data collected from a number of sensors with the support of Artificial Intelligence (AI) to enable

taking action based on the condition before a failure occurs and consequently help in extending the operational life of the equipment [1,2]. In fact, the sensor network of Industrial Internet of Things (IoT) has contributed a lot to the application of AI-based predictive maintenance in various areas, including rotating machinery [3], air compressors [4], smart transportation [2], and smart grid infrastructure [5]. These problems can be tackled directly with very promising diagnostic capability by deep learning techniques like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Hybrid architectures with only light manual participation of humans [1]. However, the majority of these systems make use of infrastructure in the cloud, which, of course, causes latency in the network, necessitates constant internet connectivity, and introduces privacy issues with data; all of these are time-critical issues in many industrial applications where fault response needs to be instantaneous (in milliseconds) [5,6].

One of the most interesting on-device inference solutions is Edge Artificial Intelligence, which enables AI on edge devices (embedded platforms) in sub-50 ms, without cloud involvement. Some works in recent years have shown the practicality of edge-based anomaly detection on devices like NVIDIA Jetson Nano and Raspberry Pi using various methods of quantization that can achieve up to 76% inference time savings and maintain the accuracy of models [4,7]. These deployments are left relatively untouched, only in the special border cases of the extremely constrained microcontrollers, such as the ESP32-S3 with just 512 KB of SRAM, for full predictive maintenance pipeline implementations, such as simultaneous fault classification and regression on the Time-to-Failure (TTF). For dynamic requirements, the existing hybrid edge-cloud systems [6] are based on static rules for offloading and cannot be dynamically adjusted to the level of prediction confidence or fault severity, and only provide up to 60% latency and bandwidth reduction. The other more specific indicator reported in the literature is the lack of an explainability approach, e.g., avoiding the use of techniques like LIME and Shapley Additive exPlanations (SHAP) to explain the decisions of the model in a predictive maintenance application for rotating machines [3] or for an air compressor system [4], or implementing such an approach in resource-constrained hybrid edge-cloud pipelines under real-time constraints.

To address these limitations, this paper provides for the first time a light-weighted, confidence-aware hybrid edge-cloud predictive maintenance pipeline, particularly designed for the ESP32-S3 microcontroller (ESPRESSIF, Riga, Latvia). The novel contribution is the development of a full and functioning AI-based predictive maintenance pipeline that can be executed deterministically on a microcontroller priced under 20 euros. The primary objective for the Edge is the ability to have as simple model as possible, as more complex models than the one architecture shown here lead to larger latencies >100 ms or consume more SRAM, which would make real-time operation impractical. Considering the hardware constraints, the INT8-Quantized edge model shows a mean latency for inference of 42.3 ms for conducting simultaneous fault detection and Time-to-Failure (TTF) estimation, which is much below the 100 ms real-time requirement. Uncertain predictions are passed to a more scalable cloud model, with the result that 97.3% of the accuracy gain of the cloud was recovered with a deterministic edge latency. Shapley Additive exPlanations (SHAP) based explanations further increase operator usability and system transparency, as do the natural-language maintenance recommendations. The full testing and hardware validation executed, including graceful degradation of safety-critical fault recall of above 0.924 even with up to 50% network outages, show that the proposed architecture is a viable solution for implementation in real-world industrial environments.

In this paper, we present a lightweight, confidence-aware hybrid edge-cloud framework for the ESP32-S3 microcontroller for predictive maintenance. It features a fully deterministic pipeline with a confidence- and latency-driven offloading mechanism that recovers 97.3% of the accuracy of the cloud model. An asymmetric loss function supports conservative fault warnings. The paper is organized as follows: Section 2 reviews related work. Section 3 details data collection, preprocessing, and feature engineering. Section 4 describes the model architecture and training. Section 5 evaluates system performance and reliability. Section 6 discusses the accuracy of deployed models under platform constraints, and Section 7 concludes and discusses directions for future research.

2. Related Works

Predictive maintenance is a key capability in industrial automation, enabling condition-based monitoring and early fault prediction to reduce downtime and maintenance costs. With advances in AI, sensor-driven diagnostics have become increasingly effective for identifying complex fault patterns beyond traditional rule-based methods. Recent work has also focused on deploying ML on resource-constrained devices. TinyML enables inference on microcontrollers with tight memory and computing budgets. A blending-based ensemble methodology [8] for predictive maintenance has been proposed that uses the A141 dataset. The proposed framework combines several base learners, such as Support Vector Machine (SVM), Random Forest (RF), Extreme Gradient Boosting (XGB), Decision Tree (DT), K-Nearest Neighbors (KNN), and Logistic Regression (LR), which is used as a meta-learner to get the ultimate predictive output. Diverging from the traditional stacking approach, the blending approach involves training the meta-model on a holdout subset, which makes the whole model construction process much simpler. Empirical evaluations show that the blending model achieves 98.8% accuracy, demonstrating improved performance in identifying machine failures [8].

However, demonstrating performance under realistic operating conditions remains a challenge. The edge-based predictive maintenance system in Reference [9], validated on an oil-injection screw compressor, performs both anomaly detection and fault classification directly on edge hardware and remains functional during network interruptions, supporting the feasibility of edge-first deployments for industrial monitoring [9].

To improve reliability while controlling latency and bandwidth, hybrid architectures that distribute computation across edge, fog, and cloud layers have been proposed. The fog-assisted framework in Reference [10] assigns lightweight anomaly detection and feature processing to the edge, uses the fog layer for intermediate analytics and confidence screening, and reserves computationally intensive diagnostics and model training for the cloud on demand. This confidence-filtering design improves responsiveness and robustness for rotating-machine monitoring [10].

Learned task offloading provides an alternative to fixed offloading rules in Industrial IoT. In reference [11], predictive-maintenance offloading is formulated as a continuous-control problem and optimized using DDPG, with an ABC-based initializer to accelerate convergence. The resulting DDPG–ABC policy outperforms greedy, random, and threshold baselines by reducing overall system cost while maintaining diagnostic accuracy under compute and communication constraints [11].

Recent works have explored TinyML and embedded neural inference for predictive maintenance on resource-constrained devices. In reference [12], a TinyML-based approach was proposed for anomaly detection in periodic duty cycles of industrial machines. Reference [13] presented a feature-based TinyML model for machine condition monitoring using edge computing. In reference [14], an AI-powered embedded framework was developed for predictive maintenance in Industrial IoT. However, these implementations mainly

focus on anomaly detection or single-task classification and rarely address simultaneous fault classification and Time-to-Failure regression under strict real-time constraints on ultra-low-cost hardware such as the ESP32-S3.

Despite these advances, gaps remain for ultra-low-cost microcontroller deployments. Much of the literature targets comparatively powerful edge platforms, and existing offloading methods often omit confidence/severity-aware decision logic and lightweight uncertainty estimation suitable for real-time inference. The present work addresses these limitations through a confidence-aware hybrid edge-cloud predictive maintenance framework tailored to ESP32-S3-class devices, enabling most diagnostics on-device while escalating uncertain cases for cloud-based analysis and reporting.

3. Data Collection, Pre-Processing, and Feature Engineering for Predictive Maintenance

Predictive maintenance is a sensor- and AI-driven approach for monitoring asset health and anticipating faults to reduce unplanned downtime, lower maintenance costs, and extend equipment lifetime. Its effectiveness depends primarily on the data pipeline, since industrial sensor streams are noisy, high-volume, and influenced by changing operating conditions (e.g., load); consequently, dataset preparation and preprocessing are typically the most time-consuming stages. In this study, synchronized multi-sensor time-series data (vibration, temperature, current, humidity, and related variables) were used across diverse regimes augmented using class weighting and targeted techniques to address class imbalance and simulate rare, physically plausible fault events, ensuring robust training while preserving the physical realism of the fault patterns required for reliable edge deployment. The resulting balanced multi-class dataset covers bearing faults, overheating, load imbalance, blockages, electrical surges, and composite faults, and is suitable for training and embedded evaluation. Benchmarking was performed against the SCANIA Component X dataset [15], whose multivariate maintenance time-series supports validating the overall trends and distributions of the constructed data, despite not explicitly labeling overheating or surge events.

3.1. Data Preprocessing and Feature Preparation

Raw sensor channels exhibit heterogeneous scales and occasional outliers; therefore, a standardized preprocessing pipeline was applied. The dataset was partitioned using a class-balanced 80/20 train–test split,

$$\text{Split Ratio} = 0.8 \text{ (Train)}: 0.2 \text{ (Test)} \tag{1}$$

Stratified sampling was applied to maintain proportional class representation across both subsets, minimizing evaluation bias in the presence of imbalanced class distributions [16]. To capture short-term signal dynamics, rolling statistical features including mean, standard deviation, maximum, and vibration magnitude were computed using 5 ms windows at a 10 kHz sampling rate. This approach facilitates the identification of both abrupt anomalies and gradual degradation in the data. All features were normalized to the range using Min–Max scaling to prevent features with larger magnitudes from biasing the model and to enhance training stability. Fault labels were encoded as integers, and class weights were calculated to address class imbalance in the dataset.

$$w_i = \frac{n_{samples}}{n_{classes} \cdot n_i} \tag{2}$$

where w_i is the weight of class i and n_i is the number of samples in class i [17].

Figure 1 presents each sensor channel as a separate histogram, including vibration axes (Vibration_X, Vibration_Y, and Vibration_Z), thermal (Temperature), electrical (Current), environmental (Humidity and Sound_Level), and prognostic (TTF_Hours) channels. The x-axis corresponds to the Min–Max normalized range [0, 1], while the y-axis denotes the percentage of samples per fault class. These distributions show clear inter-class separability across the sensor channels, validating our preprocessing pipeline.

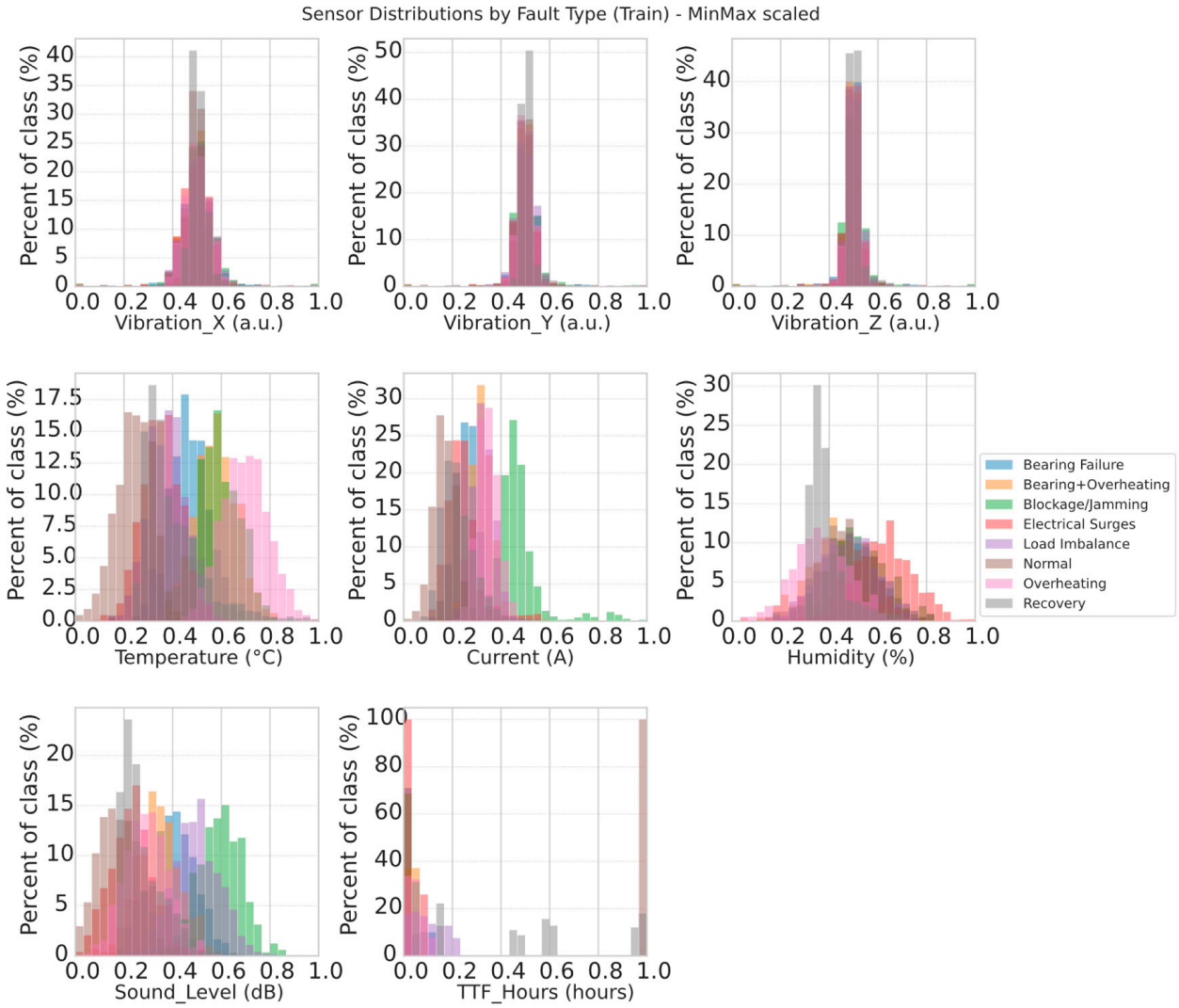


Figure 1. Min–Max normalized distributions of raw sensor features by fault class across the training set.

Pearson correlation coefficients were calculated to examine linear relationships between features:

$$r_{xy} = \frac{cov(x,y)}{\sigma_x \sigma_y} \tag{3}$$

where r_{xy} is the correlation between features x and y, and σ_x, σ_y are their standard deviations [18]. Figure 2 shows the correlation matrix of the normalized features. Highly correlated features provide redundant information, whereas weakly correlated ones offer independent predictive power, guiding the selection of optimal features for training.

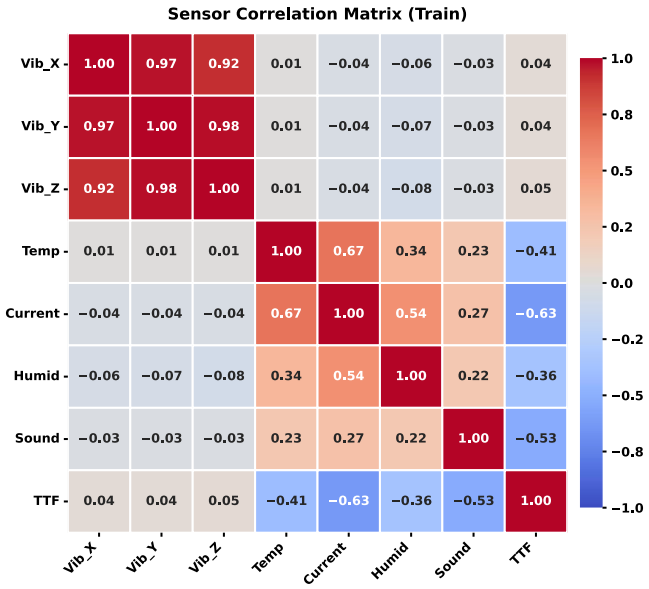


Figure 2. Correlation matrix of normalized features (training set).

3.2. Feature Engineering

Additional features were derived to expose fault-relevant patterns not evident in raw sensor signals. As discussed in References [19,20], proper feature engineering captures hidden fault patterns, improving model performance while reducing the risk of overfitting.

As shown in Figure 3, features such as TTF_trend and is_recovery provide clearer class separation, while vibration_kurtosis and current_variance exhibit weaker but still informative trends; deviations in vibration_PeakFreq and elevated surge counts indicate specific mechanical or electrical defects.

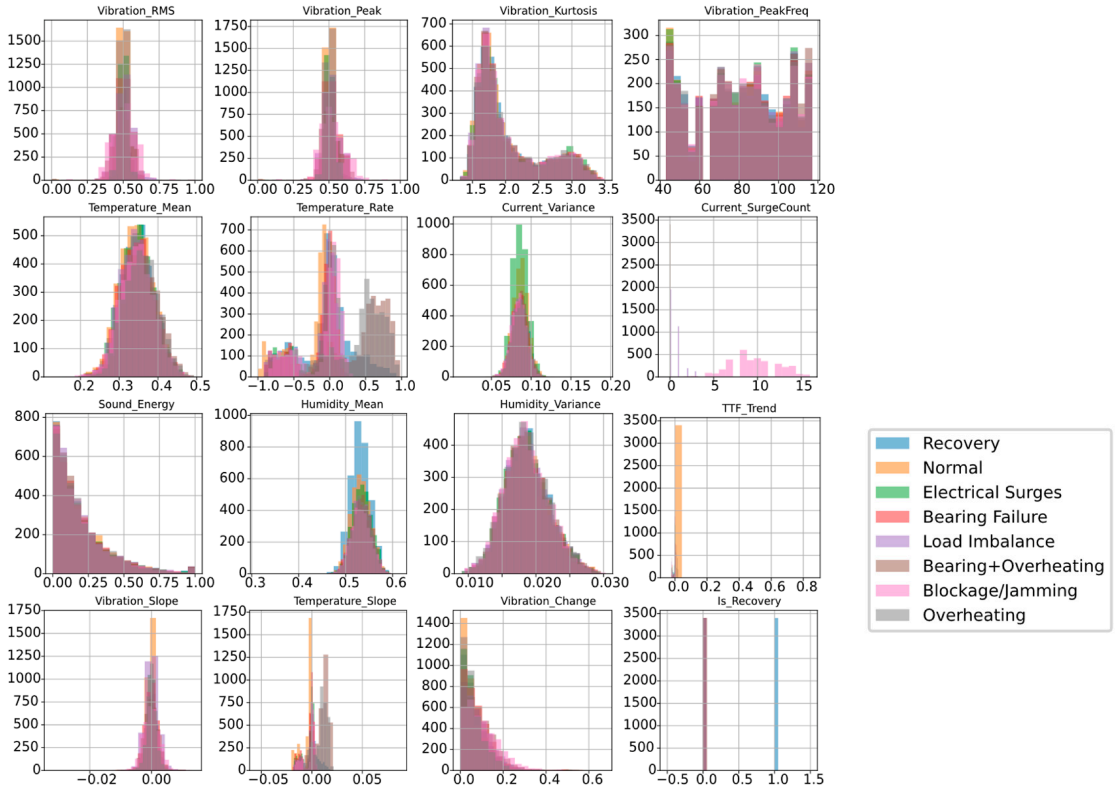


Figure 3. Distributions of engineered features by fault class, illustrating class discriminability and distinct fault signatures across statistical, frequency-domain, and temporally derived features.

4. Model Architecture and Training

Since model performance relies heavily on input data quality, we prioritized class balancing and feature engineering as our initial design steps [16]. Signal-processing and multivariate/statistical-learning principles are applied to address sensor noise, operating-condition variability, and fault interactions, while interpretable architecture and feature-importance measures improve industrial usability.

A hybrid edge-cloud architecture (Figure 4) is adopted to balance latency, reliability, and compute constraints: a lightweight ESP32-S3 edge model performs real-time fault detection and Time-to-Failure (TTF) estimation, while low-confidence cases are selectively offloaded to a higher-capacity cloud model for deeper diagnostics, consistent with scalable and energy-efficient Edge AI deployments [21,22].

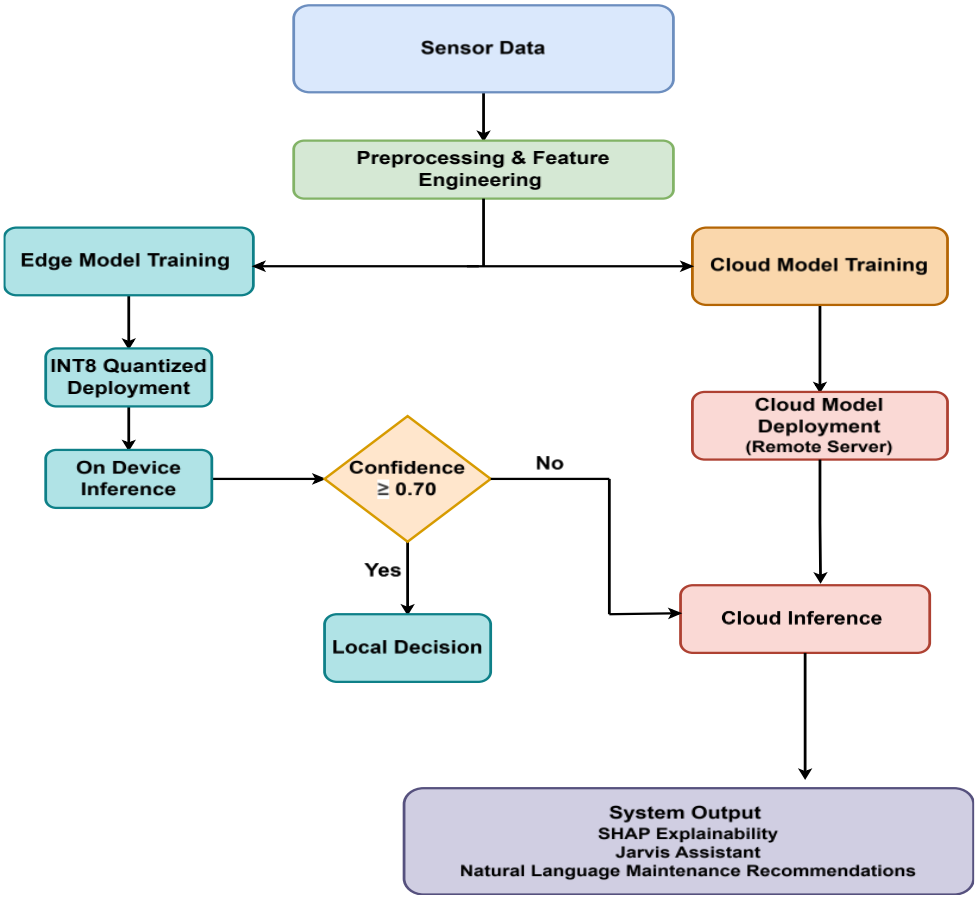


Figure 4. System architecture.

For the edge model, we utilize a parameter-efficient feed-forward network that processes the 17 engineered inputs. Hidden layers employ ReLU.

$$g(z) = max(0,\ z) \tag{4}$$

where z is the input to the neuron [23].

To mitigate overfitting, dropout regularization with a rate of 0.10 is applied after the hidden layer:

$$y = m \odot h \tag{5}$$

where h is the hidden layer output and m is a binary dropout mask [23].

For binary fault classification, a sigmoid activation function is used at the output layer:

$$\hat{y} = \sigma\left(W^T h + b\right) \tag{6}$$

where W is the weight vector, b is the bias term, and σ denotes the logistic sigmoid function [23].

To address predictive maintenance requirements, a custom asymmetric loss function is introduced for TTF estimation. Overestimating remaining useful life is penalized more heavily than underestimating:

$$\begin{cases} \alpha\left(y_{pred} - y_{true}\right)^2, & if\ y_{pred} > y_{true} \\ \left(y_{pred} - y_{true}\right)^2, & otherwise \end{cases} \tag{7}$$

where y_{pred} is the predicted Time-to-Failure, y_{true} is the actual value, and $\alpha > 1$ penalizes overestimation more than underestimation to promote early, cautious warnings.

Sample weighting further prioritizes imminent failure cases:

$$w_i = \begin{cases} 3, & y_{TTF,i} < 0.2 \\ 1, & otherwise \end{cases} \tag{8}$$

This approach increases sensitivity to critical maintenance conditions. Stratified K-fold cross-validation is applied to preserve class distributions and improve generalization across imbalanced fault categories [24]. Early stopping with a patience of 200 epochs monitors validation loss to prevent overfitting. An exponential learning rate decay schedule is used:

$$LR = LR_0 \cdot (decay\ rate)^{\frac{step}{decay\ steps}} \tag{9}$$

where LR is the current learning rate, LR_0 is the initial rate, decay rate is the decay factor, step is the current iteration, and decay steps is the interval for applying decay.

This strategy enables stable convergence and fine-grained optimization during later training stages [25]. Uncertainty estimation is achieved by keeping dropout active during inference. Multiple forward passes generate a distribution of outputs, allowing for variance-based confidence estimation. Post-training quantization converts 32-bit floating-point weights to 8-bit integers, reducing model size by approximately 70%. Accuracy degradation remains within 1–2%, while inference time is reduced to under 50 ms per sample. The edge model training process is illustrated in Figure 5. Training accuracy shows stable convergence and minimal overfitting, as presented in Figure 6.

```
Epoch 55/1000
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5871 - loss: 0.9548 - val_accuracy: 0.6029 - val_loss: 0.9142
Epoch 56/1000
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5866 - loss: 0.9590 - val_accuracy: 0.6024 - val_loss: 0.9211
Epoch 57/1000
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5839 - loss: 0.9659 - val_accuracy: 0.5984 - val_loss: 0.9178
Epoch 58/1000
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5802 - loss: 0.9621 - val_accuracy: 0.5979 - val_loss: 0.9230
Epoch 59/1000
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5849 - loss: 0.9629 - val_accuracy: 0.5959 - val_loss: 0.9228
Epoch 60/1000
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5829 - loss: 0.9719 - val_accuracy: 0.6069 - val_loss: 0.9129
Epoch 61/1000
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5834 - loss: 0.9503 - val_accuracy: 0.6089 - val_loss: 0.9160
Epoch 62/1000
250/250 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.5869 - loss: 0.9550 - val_accuracy: 0.5864 - val_loss: 0.9407
Epoch 63/1000
```

Figure 5. Shows the edge model training process.

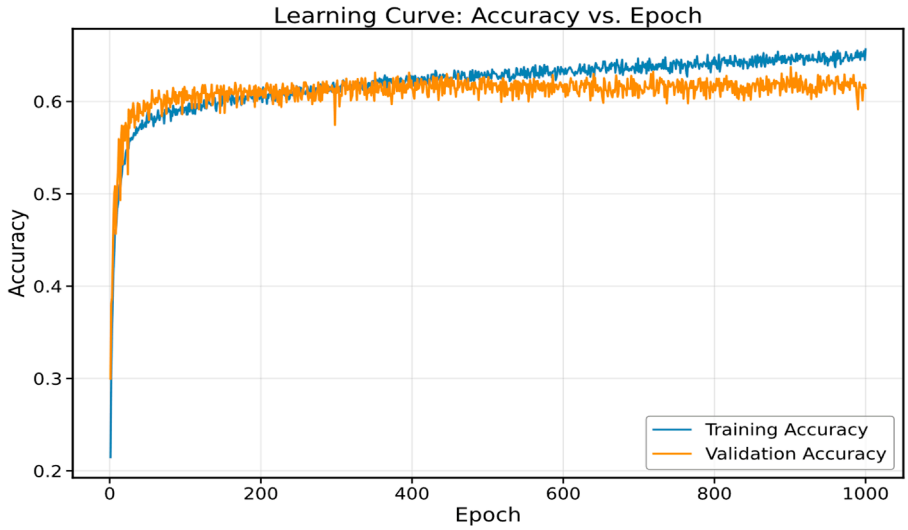


Figure 6. Illustrate Edge model accuracy during training, demonstrating stable convergence and minimal overfitting.

4.1. Cloud Model Architecture and Mechanism

Cloud architecture leverages its higher processing capacity to extract deeper features and perform complex diagnostic analysis. It has three parallel processing branches: (i) a one-dimensional convolutional neural network (1D-CNN) branch, to learn local patterns, (ii) a densely connected branch for learning global relationships, and (iii) an autoencoder branch that extracts compact latent representations. The feature maps from these three branches are concatenated to improve model robustness and generalization.

The cloud model is trained using advanced techniques, notably multitask learning, to simultaneously optimize classification and regression goals. The cloud model employs a multitask learning architecture with a shared encoder that jointly optimizes fault classification and Time-to-Failure (TTF) estimation within a single forward pass, enabling simultaneous learning of discriminative fault features and continuous degradation patterns. This joint optimization consistently outperforms single-task edge models across both classification accuracy and TTF estimation error, confirming the effectiveness of the shared representation for simultaneous diagnostic goal optimization.

Adam Algorithm [26] was used for optimization, which adjusts the learning rates based on first- and second-moment gradient estimates:

$$m_t = \beta 1 m_{t-1} + (1 - \beta_1) g_t \tag{10}$$

$$v_t = \beta_2 v_{t-1} {g_t}^2 \tag{11}$$

$$\hat{m}_t = \frac{\hat{m}_t}{1 - \beta_1^t} \tag{12}$$

$$c = \frac{v_t}{1 - \beta_2^t} \tag{13}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \in}} \tag{14}$$

where $\hat{m}_t$—first moment estimate, v_t—second moment estimate, g_t—gradient at iteration t, θ_t—parameter vector at iteration t, β_1—exponential decay rate for the first moment, β_2—exponential decay rate for the second moment, η—learning rate, $\hat{m}t$—bias-corrected first moment estimate, $\hat{v}_t$—bias-corrected second moment estimate [26], utilizing standard parameter settings $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\eta = 0.001$. A fixed batch size is maintained

throughout training. Metrics such as binary accuracy, precision, and recall are used to assess performance. The cloud model learning curve is presented in Figure 7.

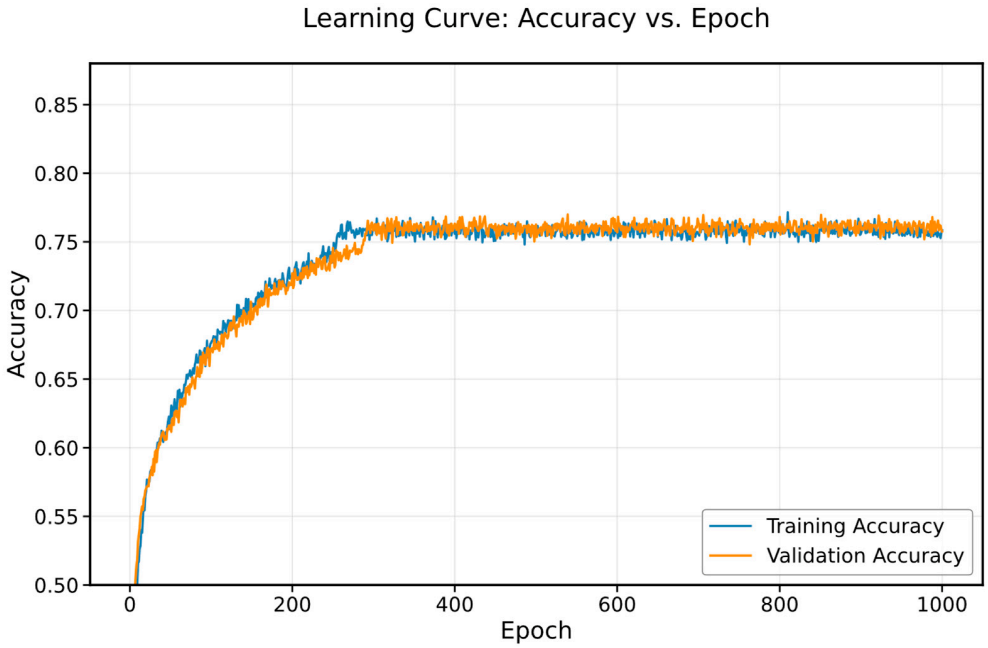


Figure 7. Shows the cloud model learning curve, indicating strong accuracy with limited overfitting.

Feature importance analysis using SHAP (Shapley Additive exPlanations) improves model transparency. By ranking the contribution of each engineered feature to the prediction, these SHAP values help operators make maintenance decisions based on the most critical sensor signals.

4.2. Combined Edge-Cloud Decision Fusion

The architecture combines both models in the decision-making process. During decision fusion, the outputs of the edge and cloud models are aggregated using a conservative, confidence-aware weighted policy. This maximizes detection sensitivity and ensures early warnings are issued with high confidence:

$$P_{combined} = \max\left(P_{edge,i}, P_{cloud,i}\right) \tag{15}$$

where $P_{combined}$ is the Combined Predicted Probability, and $P_{edge,i}$ and $P_{cloud,i}$ are the predicted probabilities from the edge and cloud model for class (i).

TTF estimation uses a minimum strategy:

$$\text{TTF(combined)} = \min\{\text{TTF(edge)}, \text{TTF(cloud)}\} \tag{16}$$

where TTF Combined is the combined estimate of Time-to-Failure, TTF(edge), and TTF(cloud) are estimates of Time-to-Failure from the edge and cloud model.

The proposed hybrid architecture differs from standard edge-cloud IoT pipelines by employing confidence-aware weighted fusion. High-confidence predictions are processed locally on the edge, while only uncertain cases are offloaded to the cloud. The system automatically falls back to the INT8-quantized edge model when the cloud is unavailable, ensuring deterministic low-latency operation and minimal network traffic.

Figure 8 shows the disagreement cases between the edge (FP32) and cloud models. The weighted fusion policy (W_cloud, W_FP32, W_INT8) resolved 55 cases correctly in favor of the true label while misclassifying only 29 cases. To explore alternative fusion

strategies and different weighting schemes, a full threshold sweep from 0.0 to 1.0 was performed. The trade-off between accuracy and latency across confidence thresholds is illustrated in Figure 9.

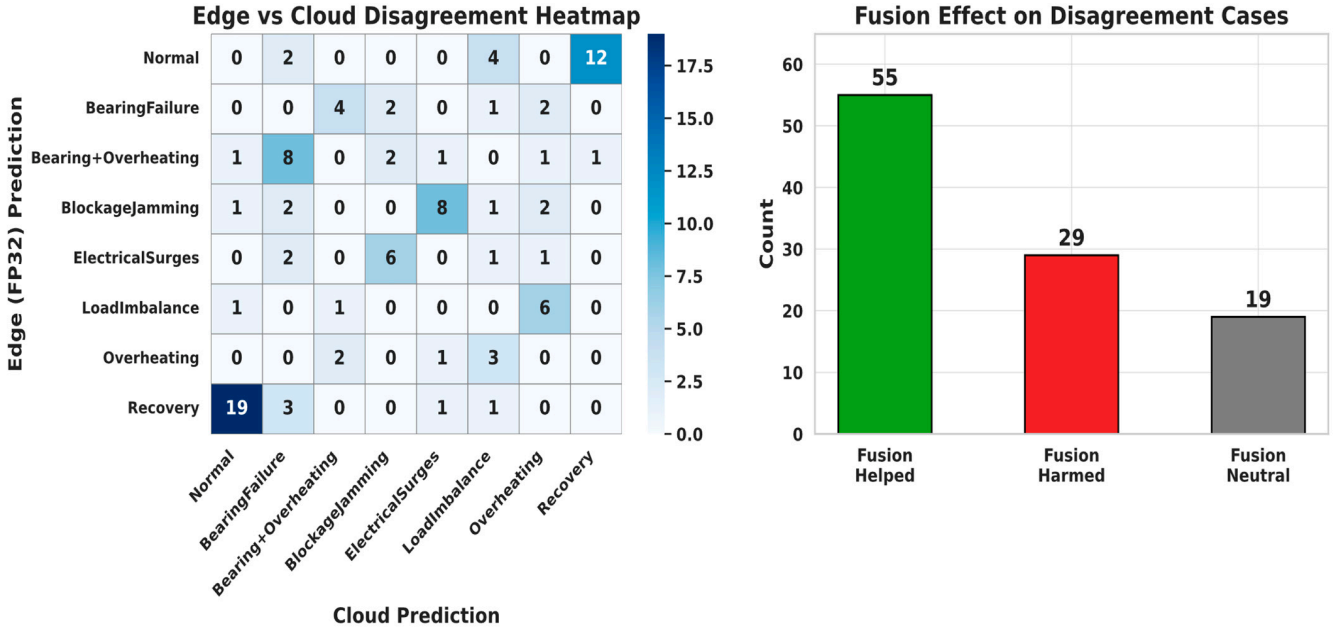


Figure 8. Edge vs. Cloud disagreement heatmap (**left**) and effect of the weighted fusion policy on disagreement cases (**right**).

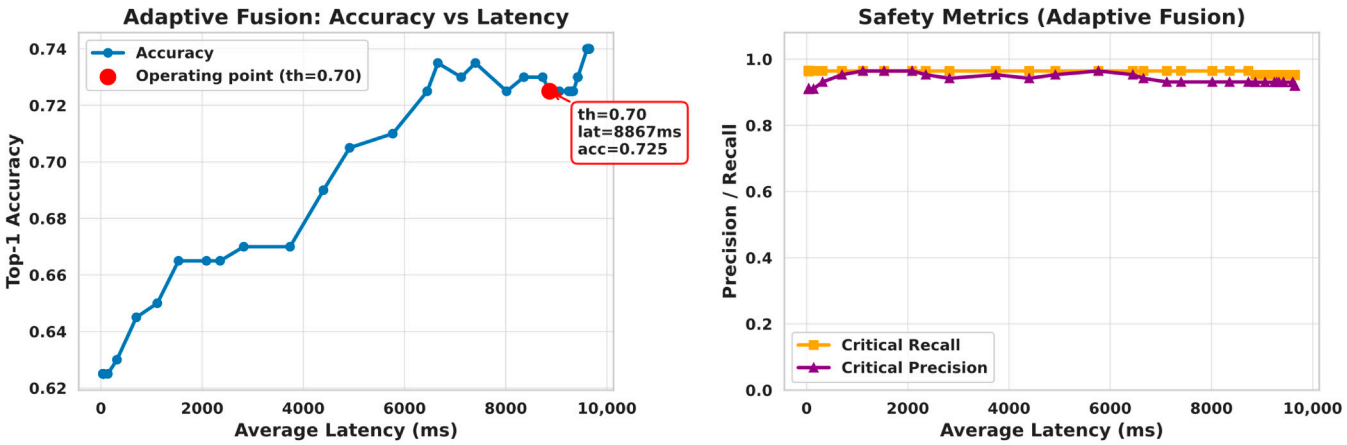


Figure 9. Adaptive fusion trade-off: Top-1 accuracy versus average latency (**left**) and safety-critical metrics (**right**).

At the chosen operating point (confidence threshold = 0.70), the hybrid system achieves 72.5% top-1 accuracy with an average latency of 8.867 ms, recovering 97.3% of the cloud accuracy gain while using only 92% of the cloud latency budget. This adaptive fusion approach balances accuracy, detection sensitivity, and real-time requirements for safety-critical industrial applications.

5. Performance and Reliability Assessment

We evaluated performance using technical and operational metrics. This ensures the system reliably detects faults and provides useful prognostic data for industrial applications. Neural network scores often represent raw confidence estimates rather than well-calibrated probabilities; procedures of calibration attempt to determine how well the estimates made

by the neural-network model predict empirical event frequencies (often visualized using reliability diagrams). A high probabilistic calibration is confirmed by the fact that the edge-model calibration curve (Figure 10) is very close to the ideal diagonal one. Deviations above or below this diagonal indicate underconfidence or overconfidence, respectively. The typical results of such classes, such as load imbalance, indicate that little training data and class imbalance contribute to the inability to predict such faults accurately [27].

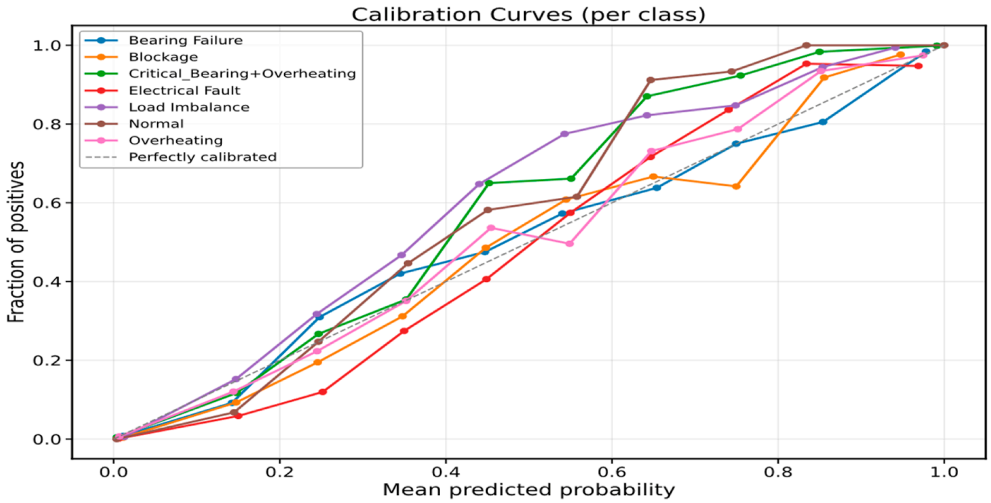


Figure 10. Calibration curve for multiclass classification (edge model).

Detection lead time measures how early the system triggers a warning before a failure. Because the system can predict failures hours or days in advance, operators can proactively schedule maintenance to prevent equipment downtime. Predicted uncertainty and absolute error result in correlation analysis, further supporting the prognostic accuracy of the model, as shown in Figure 11.

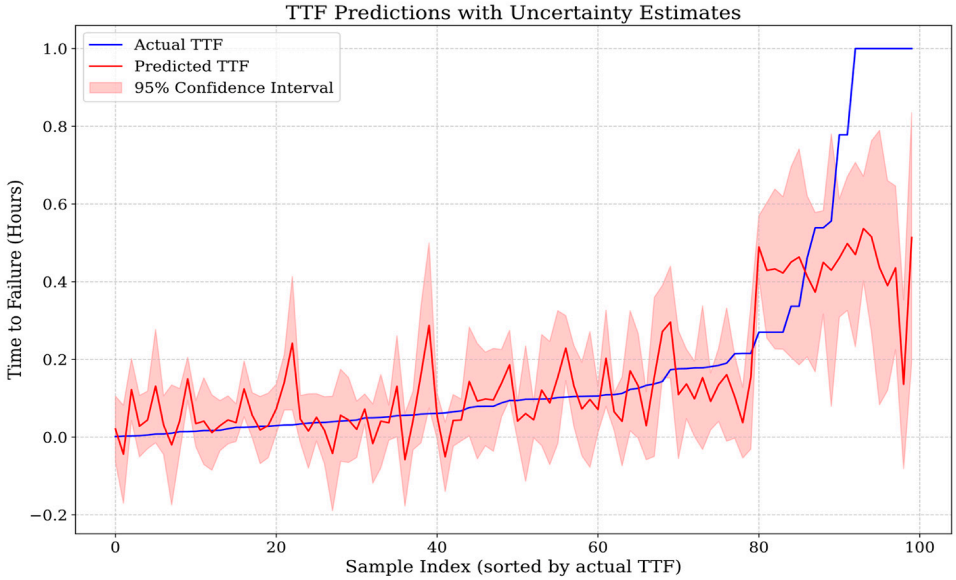


Figure 11. TTF (Time-to-Failure) predictions with uncertainty estimates.

5.1. Quantitative Performance Analysis of Neural Inference on ESP32-S3

An ESP32-S3 microcontroller (dual-core Xtensa LX7, 240 MHz, 512 KB SRAM, manufactured by ESPRESSIF, Riga, Latvia) was configured to run the quantized INT8 neural network, and either does not require or optionally uses a cloud backend running to execute

at a higher capacity. The pipeline converts the model to TensorFlow Lite Micro and deploys it via the ESP-IDF platform, utilizing static tensor arena allocation to ensure deterministic execution for real-time requirements. The confidence-based edge-cloud approach combines local predictions with cloud predictions for uncertain cases. Using a weighted fusion rule, this improves accuracy while maintaining low latency [28,29].

The model, consisting of dense layers for fault classification and TTF regression, was converted to TFLM format and quantized to INT8. Batch normalization layers were folded, and temporal convolutions replaced sequence operations for ESP-NN compatibility.

$$C_{INT8} = \sum_{l=1}^{L} \frac{W_l H_l D_l \cdot 8}{32} \cdot C_{float32} \tag{17}$$

where *CINT*8 is the computational complexity for the INT8 quantized model, L is the number of layers, and *Wl*, *Hl*, and *Dl* represent the layer dimensions. The factor of 8/32 accounts for the reduction from 32-bit to 8-bit operations. INT8 quantization is applied during model conversion, enabling computational feasibility on the MCU with minimal accuracy loss [30]. Resource viability is confirmed by flash and RAM profiling (Figure 12), which demonstrates stable on-device inference with a minimal tensor arena footprint.

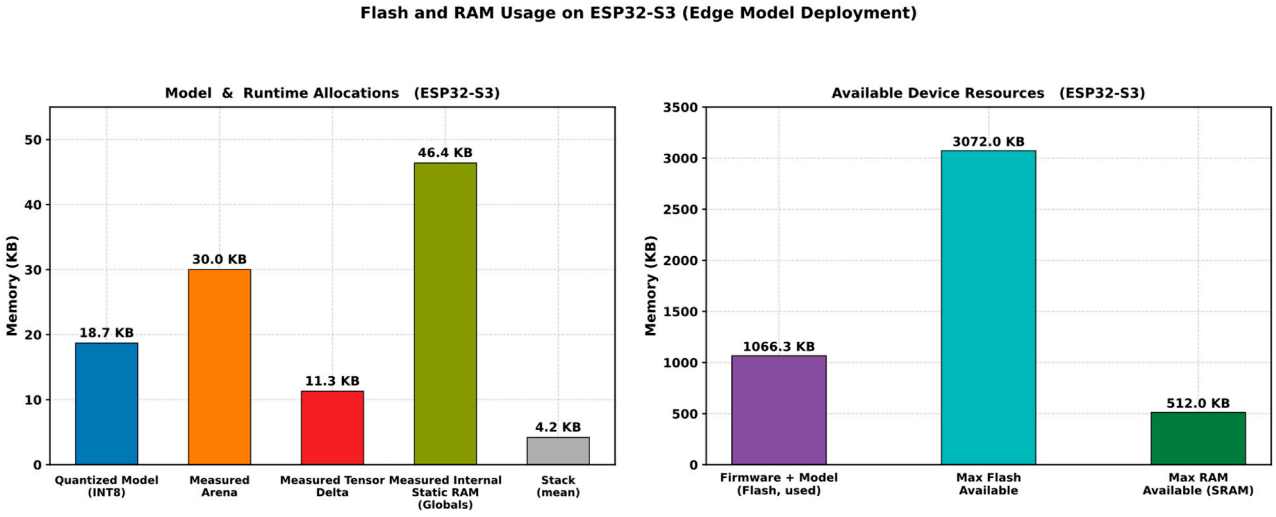


Figure 12. Flash and RAM usage on the ESP32-S3 (AI inference).

Microcontroller deployment requires embedding models into the firmware with fixed memory allocation. This limits post-deployment updates, aligning with standard practices in embedded-AI research [28,31].

It is important to note that the edge model was intentionally designed to be extremely lightweight to fit within the severe memory and compute constraints of the ESP32-S3 microcontroller. Increasing model complexity to achieve higher accuracy would result in a model too large or slow for practical on-device deployment. The achieved accuracies (62.5% for FP32 and 59.5% for INT8) (Figure 13), are therefore satisfactory for a real-time, resource-constrained TinyML application. The cloud model, being more powerful, reaches 74.0% accuracy. Through the confidence-aware hybrid fusion, the overall system achieves 72.5% accuracy while preserving deterministic sub-100 ms latency on the edge. This demonstrates a practical balance between deployability and performance.

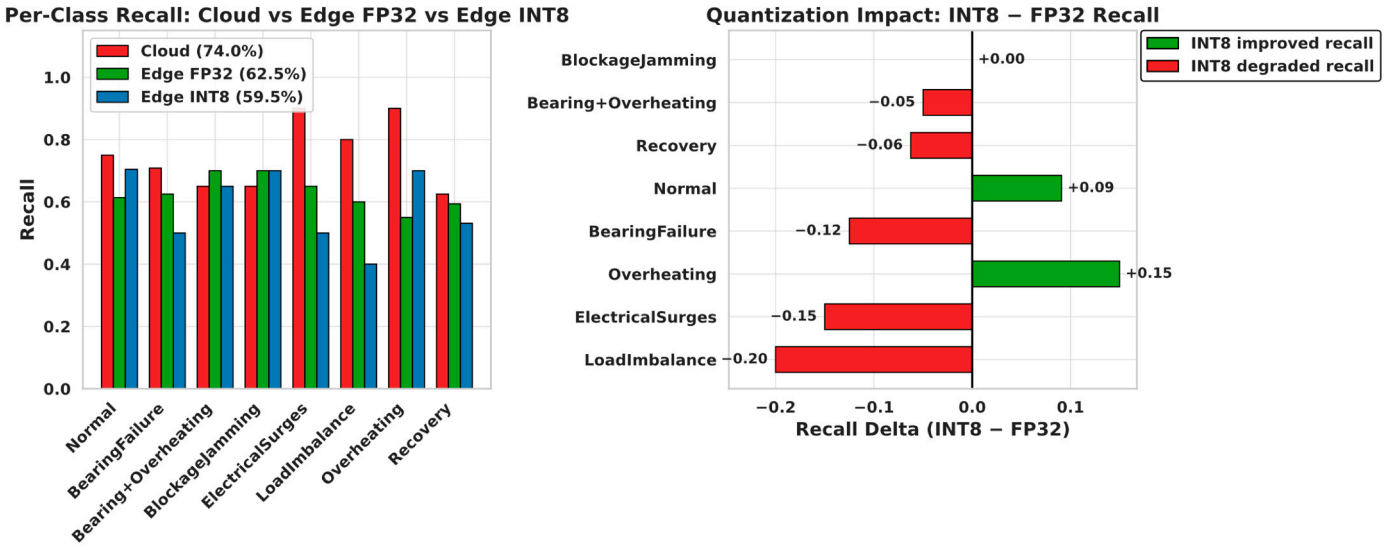


Figure 13. Per-class recall for Cloud, Edge FP32, and Edge INT8 models (**left**) and recall delta after INT8 quantization (**right**). Load Imbalance shows the largest degradation (−20%), while Overheating improves (+15%). Overall accuracy drop is only 3%.

Furthermore, bootstrap analysis validates that the performance differences between models are statistically significant, as shown by their non-overlapping 95% confidence intervals (Figure 14).

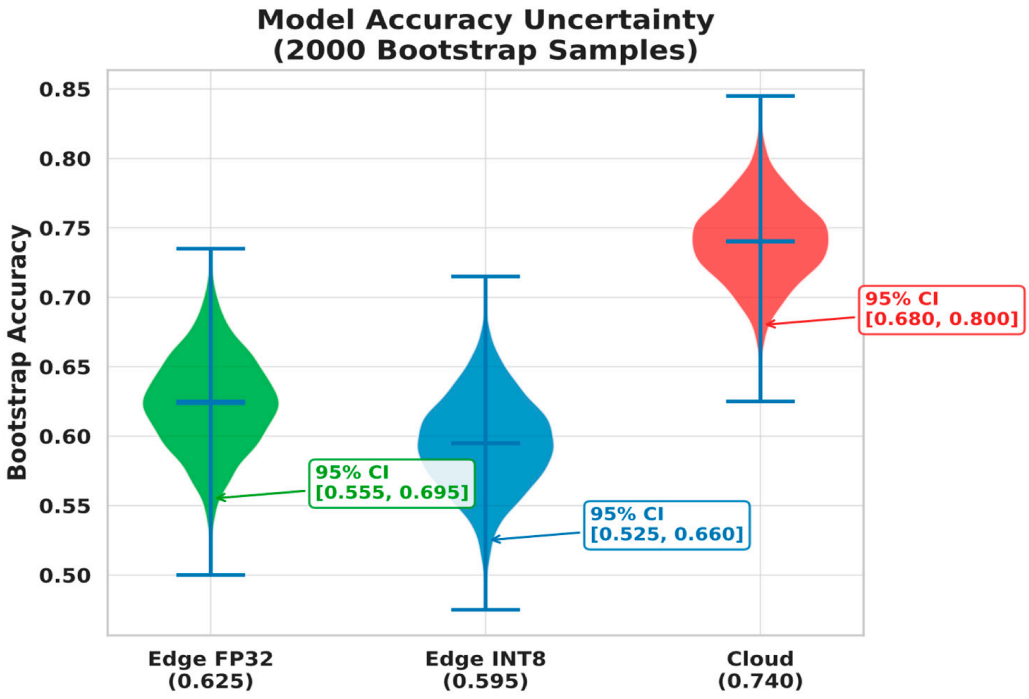


Figure 14. Model accuracy uncertainty.

Real-world deployment results on the ESP32-S3 demonstrate deterministic edge inference with a mean latency of 42.3 ms (Figure 15).

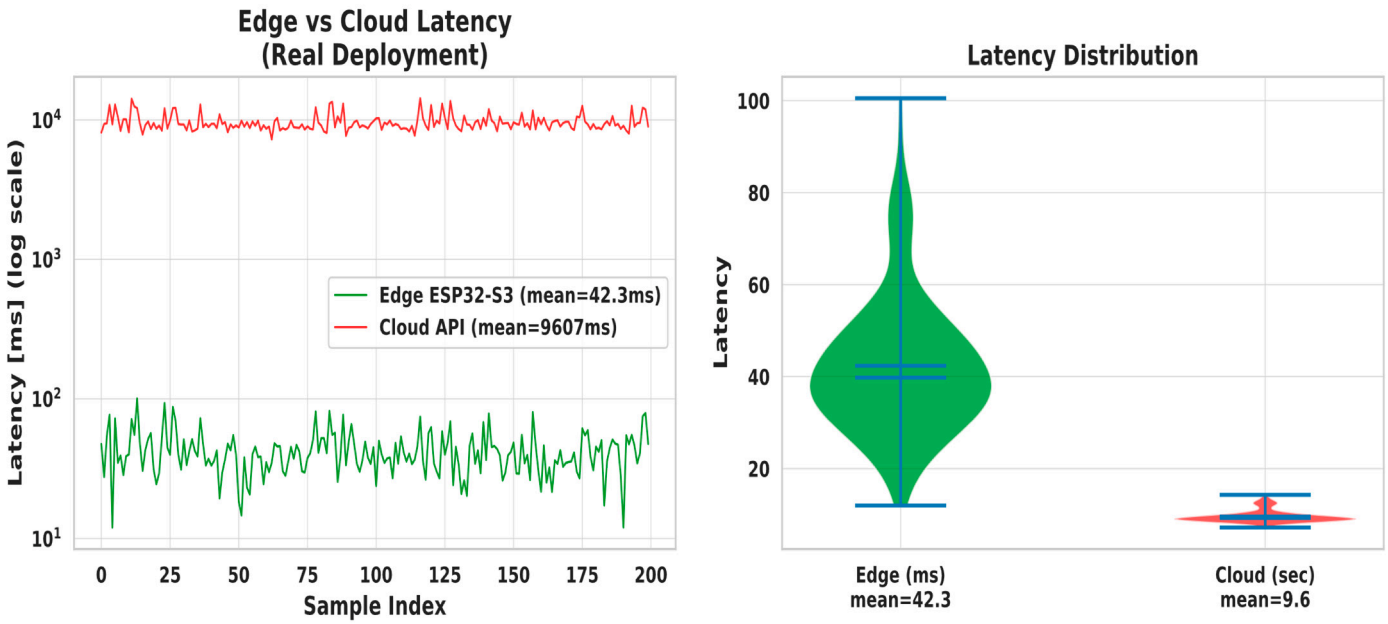


Figure 15. Edge vs. Cloud Latency (real deployment).

On-device inference achieves a deterministic latency of under 100 ms with minimal variation. In contrast, cloud inference latency is significantly higher and more variable. This aligns with recent findings in embedded systems for time-critical applications [32].

Local inference causes short CPU utilization peaks from INT8 computations, followed by rapid recovery. Cloud offloading simply shifts the workload from local processing to network communication (Figure 16). This illustrates a clear trade-off between latency and computational load.

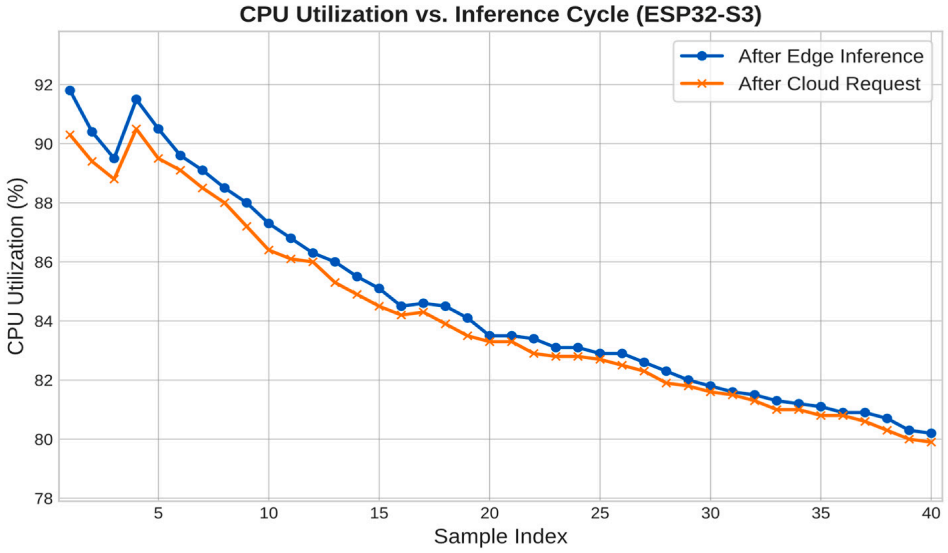


Figure 16. CPU utilization during inference cycles on ESP32-S3.

Empirical tests show that CPU activity remains stable across repeated system reboots, and post-restart jitter is minimal (Figure 17). This stability indicates proper system initialization, confirming the setup is suitable for long-term embedded applications.

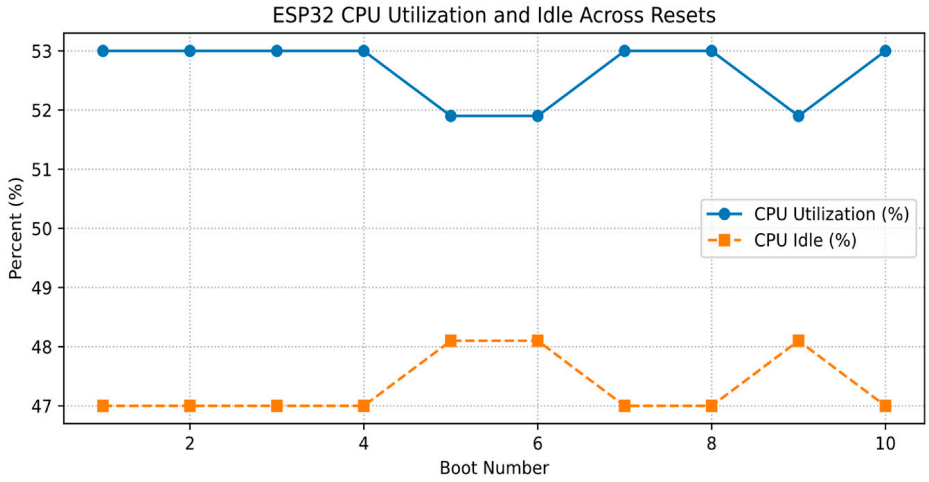


Figure 17. Stability of processing activity and idle states through system reboots.

These results demonstrate that quantized edge inference can provide robust, low-latency operation within the confines of an MCU, and cloud inference continues to be beneficial as an auxiliary diagnostic route. Previous studies on vehicle compliance monitoring support the feasibility of real-time edge inference [33]. The findings are aligned with memory-based edge-AI studies and associated deployment studies [28,29], and deployment evidence based on applications, i.e., bearing diagnostics and optimized inference deployments [30,34,35].

5.2. Robustness to Varying Network Connectivity and Latency

To test system robustness, we simulated network outages with drop rates from 0% to 50%. During these outages, the system automatically falls back to the local INT8-quantized edge model.

The results, as shown in Figure 18, show that the system maintains strong safety-critical performance even when the cloud fails for half of all requests. The conservative INT8 fallback policy allows the edge model to reliably detect critical faults on its own. This confirms that the hybrid architecture is practical for industrial IoT settings, where network connections are often unreliable.

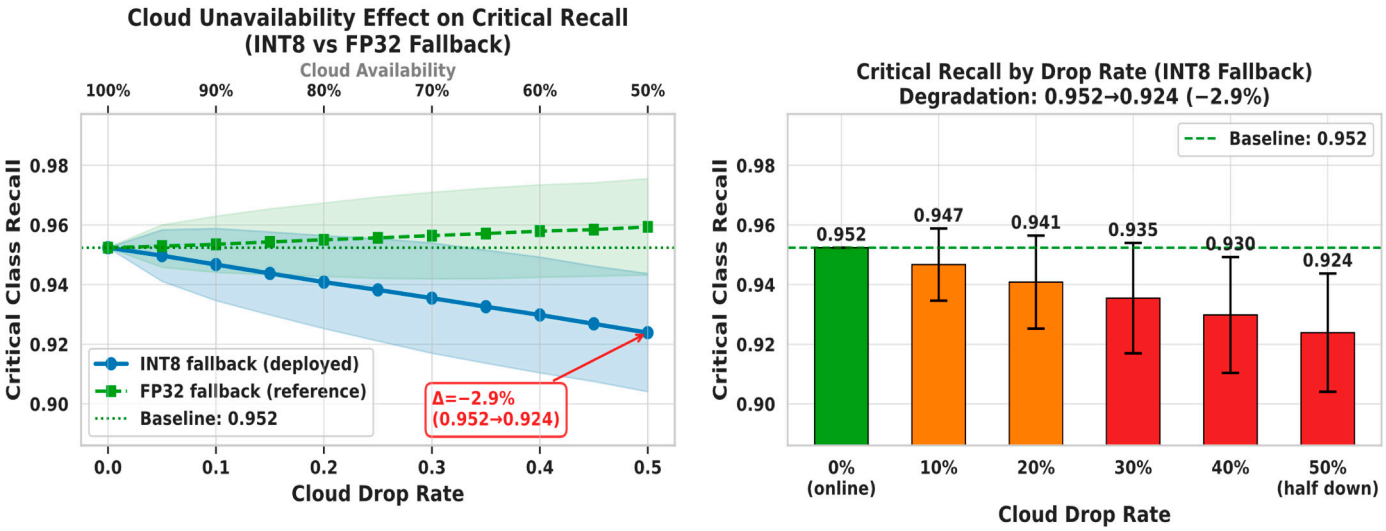


Figure 18. Cloud unavailability effect on critical recall (INT8 vs. FP32 fallback). Even at 50% cloud outage, critical fault recall remains 0.924 (only 2.9% degradation from the 0.952 baseline), demonstrating graceful degradation and acceptable safety margins for industrial deployment.

5.3. Time-to-Failure (TTF) Estimation and the Role of the Asymmetric Loss Function

The system performs fault classification and estimates the remaining useful life using a Time-to-Failure (TTF) regression head. True versus predicted Time-to-Failure values and mean absolute error (MAE) comparison across models is shown in Figure 19. During training, we applied a custom asymmetric loss function to penalize overestimations of remaining life more heavily than underestimations. This design makes the model more conservative and provides earlier warnings when critical faults are developing.

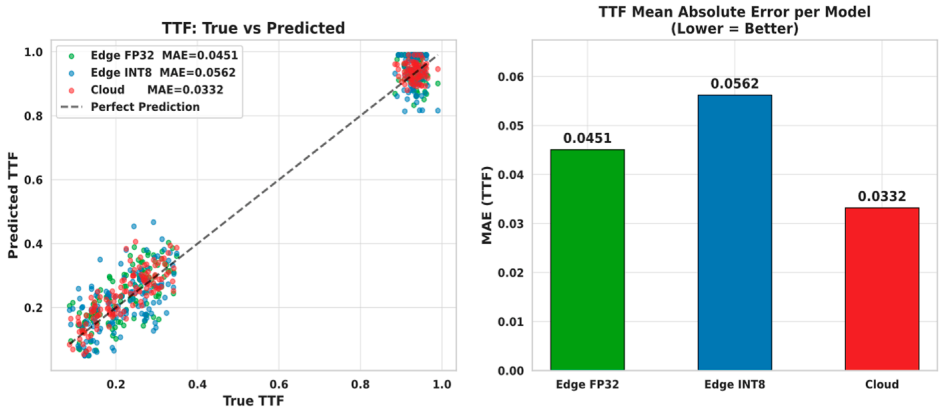


Figure 19. True versus predicted Time-to-Failure values (**left**) and mean absolute error (MAE) comparison across models (**right**). The cloud model achieves the lowest MAE of 0.0332, representing a 26.3% improvement over the edge FP32 model (MAE = 0.0451).

The results demonstrate that the asymmetric loss function increases sensitivity to critical conditions. The hybrid architecture leverages fast edge predictions for real-time decisions. It then offloads uncertain cases to the cloud for more accurate TTF estimation. The substantial reduction in MAE when using the cloud model confirms that our chosen loss function and multitask learning approach work effectively in practice and provide reliable remaining-life estimates for maintenance planning.

6. Deployed Model Under Platform Constraints

The real-time edge inference output (Figure 20) on the ESP32 via the serial monitor. The logs include multi-modal sensor data and system telemetry. This confirms that the inference process stays within operational limits and allows for easy performance tracking.

```
Waiting for prediction...

========== ESP32-S3 AI Dashboard ==========
╔ WiFi: Connecting •••••• Connected ╗
╔═══ NEW INPUT ═══╗
╔═══ EDGE Features ═══╗
Vib_RMS    :  0.53  Vib_Peak  :  0.54  Vib_Kurt  :  0.56  Vib_PkFr  :  0.79
Tmp_Mean   :  0.08  Tmp_Rate  :  0.08  Cur_Var   :  0.00  Cur_SrgC  :  0.00
Snd_En     :  0.26  Hum_Mean  :  0.77  Hum_Var   :  0.00  TTF_Trnd  :  0.92
Vib_Slp    :  0.38  Tmp_Slp   :  0.57  Vib_Chg   :  0.10  Is_Rec    :  0.00
Tmp_AbnD   :  0.00  ╚═══════════════════════════════════════════════╝
╔═══ EDGE AI PREDICTION ═══╗
Bearing Failure          2.73%
Blockage                37.89%
Critical_Bearing+Overheating   0.39%
Electrical Fault        19.53%
Load Imbalance          19.53%
Normal                  19.53%
Overheating              0.00%
► Predicted: Blockage   Conf: 37.89%
╚═══════════════════════════╝
«EDGE» Heap: 8445k Min: 8435k | Stack: 5628
╔═══ NEW INPUT ═══╗
╔═══ CLOUD Features ═══╗
Vib_RMS    :  0.53  Vib_Peak  :  0.54  Vib_Kurt  : -1.50  Vib_PkFr  :347.66
Tmp_Mean   :  0.07  Tmp_Rate  :  0.00  Cur_Var   :  0.00  Cur_SrgC  :  0.00
Snd_En     :  0.20  Hum_Mean  :  0.55  Hum_Var   :  0.00  TTF_Trnd  : -0.00
Vib_Slp    : -0.00  Tmp_Slp   :  0.00  Vib_Chg   :  0.00  Is_Rec    :  0.00
Tmp_AbnD   :  0.00  ╚═══════════════════════════════════════════════╝
[Cloud] Sending features...
```

Figure 20. Real-time Edge inference predictions output on ESP32 (serial monitor log).

Figure 21 illustrates how the system translates raw predictions and TTF estimates into clear, natural-language maintenance recommendations. We developed a natural-language recommendation module that leverages the OpenAI GPT-5 mini API to translate raw diagnostic outputs into actionable maintenance recommendations for operators. It should be noted that this module serves as a demonstration of operator-facing explainability and introduces a dependency on a third-party commercial API, which may affect reproducibility and long-term system availability.

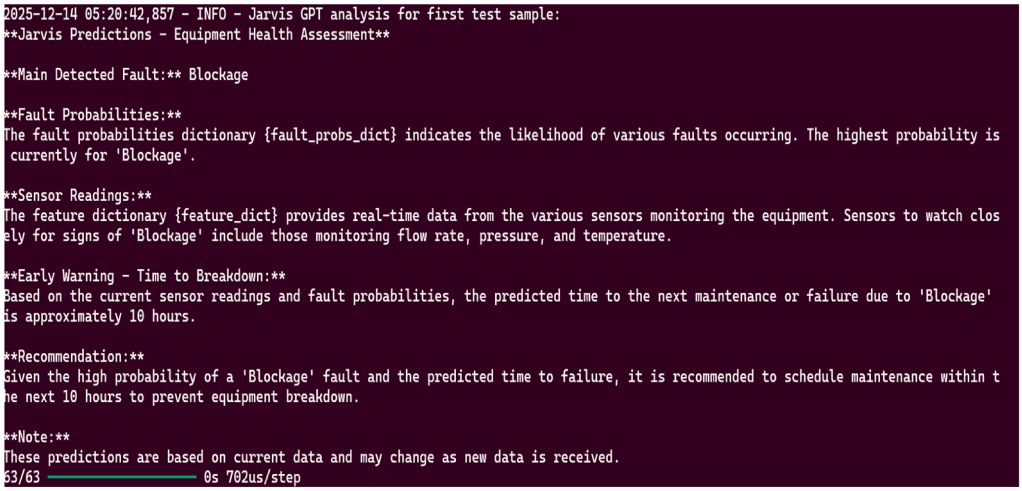


Figure 21. Jarvis in action—maintenance recommendations (cloud model).

This evaluation of memory, CPU, latency, and accuracy demonstrates that edge AI delivers dependable performance under strict hardware constraints. Even though cloud-based inference offers greater accuracy and interpretability, it has more fluctuating latency, meaning it can only serve as a complementary diagnostic role. The findings underscore the significance of controlled memory management, deterministic execution, and the sensitivity of quantization in industrial edge AI systems and hence serve as the basis of real-world implementation and future study.

Industrial Applications and Examples

The hybrid edge-cloud system developed is directly applicable in industrial environments. Similar to recent edge AI industrial automation deployments [3,9], an ESP32-S3 node can be used in smart manufacturing plants to be installed on critical rotating machinery like centrifugal pumps, conveyor systems, and induction motors. The system delivers sub-100 ms fault detection and early TTF warnings, allowing maintenance teams to plan repairs before faults occur and minimizing unplanned downtime. The INT8 fallback mechanism guarantees not to stall the system during cloud outages. It is shown in Section 5.2 that it is capable of maintaining the critical fault recall of 0.924 at the same time, even with just 50% cloud outage. The system introduces user-friendliness to operators without AI knowledge by integrating SHAP explanations and natural-language recommendations. The application scenarios also show that there are obvious operational and economic advantages in implementing the proposed low-cost, low-latency edge solution in various industrial sectors [21,22].

7. Conclusions

This paper introduces a lightweight, confidence-aware hybrid edge-cloud predictive maintenance system, which has been developed and rigorously evaluated on the ESP32-S3 microcontroller platform. Interestingly, there is no special chipset to handle AI acceleration

for ESP32-S3, as well as just 512 KB of SRAM, making it, on its own, a severe constraint compared with popular edge AI development platforms. The first goal of the work is to confirm the ability of the full and deterministic predictive maintenance pipeline for such ultra-constrained yet cost-effective embedded devices, which is in contrast with existing literature that has been able to successfully implement the same on more powerful devices such as NVIDIA Jetson Nano or Raspberry Pi [4,7].

The models on the edge and cloud have been optimized to ensure that they are fusion compatible. The intentional architectural compromise is to deliver good overall diagnostic performance, yet deterministic low-latency operation at the edge device. In particular, the INT8-trained edge model achieves an average of 42.3 ms of inference latency. The anticipated gain of accuracy by the cloud-based model, combined with the confidence-aware fusion mechanism, recovers 97.3% of the accuracy, while under 92% of the cloud latency constraint. To give a preference for conservative and early detection of faults, an asymmetric loss function is used. Moreover, SHAP and natural language recommendations keep diagnostic outputs interpretable for operators without deep AI expertise [3,4].

The system was determined to be robust through assessments and is capable of achieving a safety-critical fault recall above 0.924 even in the case of simulated network failure ratios of up to 50%. This exemplifies the graceful degradation of the system, which plays a particularly important role in applications for the field of industrial IoT, where the connection cannot be assured [5,6]. If edge and cloud are used as a single diagnostic "flow" instead of a separate "flow" there are clear mechanisms for reliable edge AI inference enabled by microcontroller-class hardware, according to the findings of this study. The interplay between the various aspects of confidence-aware offloading, asymmetric loss optimization, and graceful degradation can together tackle the need for low latency, accuracy, and connectivity, which are the major challenges in predictive maintenance technologies in industrial settings in general [1,6].

Future research directions include the scalability of expanding to multi-device field deployments, the validation of the system on public benchmarks, and researching 'federated learning' approaches to model updates for privacy-preserving model updates [7]. Furthermore, this work suggests a formal comparison of the stratified K-fold and the temporal walk-forward cross-validation strategies for future work, especially for datasets representing continuous degradation trajectories. A systematic sensitivity analysis of different window sizes and sampling rates is also recommended for future work, especially for the optimization of the detection of slow-progressing faults such as bearing wear and thermal degradation. This framework will also be customized to cover a wider class of microcontroller architectures, while paving the way towards intelligent low-cost embedded maintenance systems [1,2].

Author Contributions: Conceptualization, G.K. and M.T.; methodology, G.K. and A.A.R.; software, G.K.; validation, G.K., AAR., M.T., V.S., A.K.M.P.T. and P.E.S.; formal analysis, A.A.R. and P.E.S.; investigation, G.K. and M.T.; resources, V.S.; data curation, G.K. and A.A.R.; writing—original draft preparation, G.K.; writing—review and editing, A.A.R.; visualization, G.K.; supervision, M.T. and J.S.; project administration, M.T. and J.S.; funding acquisition, M.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Latvian Council of Science, LZP Fundamental and Applied Research Project, grant No. lzp-2024/1-0442, registration number: 4938, "Single chip-based digital oscilloscope for remote education".

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data supporting the conclusions of this article will be made available by the authors on request.

Acknowledgments: The authors acknowledge the support provided by the Institute of Photonics, Electronics and Telecommunications at Riga Technical University.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

1D-CNN	One-Dimensional Convolutional Neural Network
ABC	Artificial Bee Colony
AI	Artificial Intelligence
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DDPG	Deep Deterministic Policy Gradient
DT	Decision Tree
ESP-IDF	Espressif IoT Development Framework
ESP32-S3	Espressif Systems 32-bit Microcontroller Series 3
FP32	32-bit Floating Point
IIoT	Industrial Internet of Things
INT8	8-bit Integer
IoT	Internet of Things
KNN	K-Nearest Neighbors
LIME	Local Interpretable Model-Agnostic Explanations
LR	Logistic Regression
MAE	Mean Absolute Error
MCU	Microcontroller Unit
ML	Machine Learning
RAM	Random Access Memory
ReLU	Rectified Linear Unit
RF	Random Forest
RNN	Recurrent Neural Network
SHAP	Shapley Additive exPlanations
SRAM	Static Random Access Memory
SVM	Support Vector Machine
TFLM	TensorFlow Lite Micro
TinyML	Tiny Machine Learning
TTF	Time-to-Failure
XGB	Extreme Gradient Boosting

References

1. Kumar, V.; Prakash, M.; Thamburaj, S. Deep Learning-based Predictive Maintenance for Industrial IoT Applications. In Proceedings of the 2024 International Conference on Inventive Computation Technologies (ICICT), Lalitpur, Nepal, 24–26 April 2024; pp. 1197–1202. [CrossRef]
2. Hundekari, S.; Mittal, T.; Dutt, A.; Bhavana, M.; Kumar, R.; Nijhawan, G. AI-Driven Solutions for Predictive Maintenance in Smart Transportation Systems. In Proceedings of the 2025 International Conference on Computational, Communication and Information Technology (ICCCIT), Indore, India, 7–8 February 2025; pp. 572–577. [CrossRef]
3. Gawde, S.; Patil, S.; Kumar, S.; Kamat, P.; Kotecha, K.; Alfarhood, S. Explainable Predictive Maintenance of Rotating Machines Using LIME, SHAP, PDP, ICE. *IEEE Access* **2024**, *12*, 29345–29361. [CrossRef]
4. Ciobotaru, A.; Corches, C.; Gota, D.; Miclea, L. An Explainable Deep Learning-Based Predictive Maintenance Solution for Air Compressor Condition Monitoring. *Sensors* **2025**, *25*, 5797. [CrossRef] [PubMed]

5. Rustambekov, I.; Gulyamov, S.S.; Abduvaliyev, B.; Kan, E.; Abdukhakimov, I.; Yakubova, M.; Karimov, D. Predictive Maintenance of Smart Grid Components Based on Real-Time Data Analysis. In Proceedings of the 2024 6th International Conference on Control Systems, Mathematical Modeling, Automation and Energy Efficiency (SUMMA), Lipetsk, Russia, 13–15 November 2024; pp. 949–952. [CrossRef]
6. Sathupadi, K.; Achar, S.; Bhaskaran, S.V.; Faruqui, N.; Abdullah-Al-Wadud, M.; Uddin, J. Edge-Cloud Synergy for AI-Enhanced Sensor Network Data: A Real-Time Predictive Maintenance Framework. *Sensors* **2024**, *24*, 7918. [CrossRef] [PubMed]
7. Reis, M.J.C.S.; Serôdio, C. Edge AI for Real-Time Anomaly Detection in Smart Homes. *Future Internet* **2025**, *17*, 179. [CrossRef]
8. Salini, S.M.; Geetha, S. TinyML-Powered Predictive Maintenance Framework on Resource-Constrained Devices: A Blended Learning Approach. In Proceedings of the 2025 8th International Conference on Computing Methodologies and Communication (ICCMC), Erode, India, 23–25 July 2025; pp. 1119–1126. [CrossRef]
9. Ringler, N.; Knittel, D.; Ponsart, J.-C.; Nouari, M.; Yakob, A.; Romani, D. Machine Learning Based Real Time Predictive Maintenance at the Edge for Manufacturing Systems: A Practical Example. In Proceedings of the 2023 IEEE IAS Global Conference on Emerging Technologies (GlobConET), London, UK, 19–21 May 2023; pp. 1–6. [CrossRef]
10. Ech-Chadi, S.; Zaid, M.A.; Chadi, Y.E. A New Framework Based on FOG Computing and Edge AI to Increase the Reliability of Predictive Maintenance of Rotating Machines. In Proceedings of the 2025 International Conference on Circuit, Systems and Communication (ICCSC), Fez, Morocco, 19–20 June 2025; pp. 1–5. [CrossRef]
11. Zhang, B.; Wang, C. Deep Reinforcement Learning-based Predictive Maintenance Task Offloading and Resource Allocation. In Proceedings of the 2023 IEEE 23rd International Conference on Communication Technology (ICCT), Wuxi, China, 20–22 October 2023; pp. 659–664. [CrossRef]
12. Martinez-Rau, L.S.; Zhang, Y.; Oelmann, B.; Bader, S. TinyML Anomaly Detection for Industrial Machines with Periodic Duty Cycles. In Proceedings of the 2024 IEEE Sensors Applications Symposium (SAS), Naples, Italy, 23–25 July 2024; pp. 1–6. [CrossRef]
13. Zhao, X.; Wang, P. A Deployable Edge Computing Solution for Machine Condition Monitoring. In Proceedings of the 2024 IEEE International Instrumentation and Measurement Technology Conference (I2MTC), Glasgow, UK, 20–23 May 2024; pp. 1–6. [CrossRef]
14. Kumar, G.A.; Shobana, K.B.; Venkatesh, T.; Lekha, B.C. AI-Powered Embedded Systems for Predictive Maintenance in Industrial IoT Devices. In Proceedings of the 2025 IEEE 3rd Global Conference on Wireless Computing and Networking (GCWCN), Lonavala, India, 22–23 November 2025; pp. 1–5. [CrossRef]
15. Research Data Sweden. Dataset 2024-34/2. Available online: https://researchdata.se/en/catalogue/dataset/2024-34/2 (accessed on 10 March 2026).
16. Aruna Kumara, B.; Kodabagi, M.M. Efficient Data Preprocessing Approach for Imbalanced Data in Email Classification System. In Proceedings of the 2020 International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE), Bengaluru, India, 9–10 October 2020; pp. 338–341. [CrossRef]
17. Scikit-Learn Developers. Available online: https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html (accessed on 10 March 2026).
18. NumPy Developers. numpy.corrcoef. Available online: https://numpy.org/doc/stable/reference/generated/numpy.corrcoef.html (accessed on 10 March 2026).
19. Croft, R.; Xie, Y.; Babar, M.A. Data Preparation for Software Vulnerability Prediction: A Systematic Literature Review. *IEEE Trans. Softw. Eng.* **2023**, *49*, 1044–1063. [CrossRef]
20. Behnke, M.; Briner, N.; Cullen, D.; Schwerdtfeger, K.; Warren, J.; Basnet, R.; Doleck, T. Feature Engineering and Machine Learning Model Comparison for Malicious Activity Detection in the DNS-Over-HTTPS Protocol. *IEEE Access* **2021**, *9*, 129902–129916. [CrossRef]
21. Sevilla Martínez, F.; Casas-Roma, J.; Subirats, L.; Parada, R. Eco-Efficient Deployment of Spiking Neural Networks on Low-Cost Edge Hardware. *IEEE Netw. Lett.* **2026**, *8*, 39–43. [CrossRef]
22. Anchitaalagammai, J.V.; Kavitha, S.; Buurvidha, R.; Santhiya, T.S.; Devi Roopa, M.; Sri Sankari, S. Edge Artificial Intelligence for Real-Time Decision Making using NVIDIA Jetson Orin, Google Coral Edge TPU and 6G for Privacy and Scalability. In Proceedings of the 2025 International Conference on Visual Analytics and Data Visualization (ICVADV), Tirunelveli, India, 4–6 March 2025; pp. 150–155. [CrossRef]
23. Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*; MIT Press: Cambridge, MA, USA, 2016; Available online: http://www.deeplearningbook.org (accessed on 15 November 2025).
24. Xi, M.; Zhao, L.; Ge, J.; Chan, J.; Zhang, X.; Li, C.; Wu, Z.; Huang, Z. An Interpretable Synthesis Method of Intelligent Surface for Dynamic Antenna Isolation Control Based on SHAP Analysis. *IEEE Antennas Wirel. Propag. Lett.* **2025**, *24*, 3931–3935. [CrossRef]
25. Pandian, A.; Subbian, S.; Natarajan, P.; Deepa, S.S. Uncertainty Quantification of LSTM Model for SoC Estimation against Aleatoric and Epistemic Uncertainties. In Proceedings of the 2024 Control Instrumentation System Conference (CISCON), Manipal, India, 2–3 August 2024; pp. 1–6. [CrossRef]
26. Kingma, D.P.; Ba, J. Adam: A Method for Stochastic Optimization. *arXiv* **2014**. [CrossRef]

27. Khan, S.A.; Rana, Z.A. Evaluating Performance of Software Defect Prediction Models Using Area Under Precision-Recall Curve (AUC-PR). In Proceedings of the 2019 2nd International Conference on Advancements in Computational Sciences (ICACS), Lahore, Pakistan, 18–20 February 2019; pp. 1–6. [CrossRef]
28. Caon, M.; Choné, C.; Schiavone, P.D.; Levisse, A.; Masera, G.; Martina, M.; Atienza, D. Scalable and RISC-V Programmable Near-Memory Computing Architectures for Edge Nodes. *IEEE Trans. Emerg. Top. Comput.* **2025**, *13*, 1003–1018. [CrossRef]
29. Guo, Y.-C.; Chang, T.-S.; Lin, C.-S.; Chiou, B.-C.; Lai, C.-M.; Sheu, S.-S.; Lo, W.-C.; Chang, S.-C. CIMR-V: An End-to-End SRAM-based CIM Accelerator with RISC-V for AI Edge Device. In Proceedings of the 2024 IEEE International Symposium on Circuits and Systems (ISCAS), Singapore, 19–22 May 2024; pp. 1–5. [CrossRef]
30. Manickam, D.D.; Mohamed, S.; Jain, V.; Goswami, D.; Lensink, L. A Structured Inference Optimization Approach for Vision-Based DNN Deployment on Legacy Systems. In Proceedings of the 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA), Sinaia, Romania, 12–15 September 2023; pp. 1–8. [CrossRef]
31. Fu, H.; Deng, L.; Tang, B.; Cui, S.; Fu, Y. Neural Networks Micro Memory Control Strategy for Mechanical Faults Edge Recognition. *IEEE Trans. Ind. Inf.* **2025**, *21*, 5069–5080. [CrossRef]
32. Adam Ahmad, J.; Alasiry, A.H.; Darmawan, A.; Alfathdyanto, K.; Firdaus, R.S.; Mauludi, F. TPU-MLIR-Driven INT8 Quantization for YOLO Models: Enabling High-Performance Edge AI in Autonomous Quadruped Robots. In Proceedings of the 2025 International Electronics Symposium (IES), Surabaya, Indonesia, 5–7 August 2025; pp. 403–409. [CrossRef]
33. Abinaya, R.; Sekar, R.; Pai, T.V.; Thippa, S.; Praveen Joe, I.R. Deep Learning-Based Helmet Detection for In-Vehicle Compliance Monitoring on Low-Cost Edge Devices. In Proceedings of the 2025 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS), Bhopal, India, 18–19 January 2025; pp. 1–5. [CrossRef]
34. Ma, K.; Wu, S.; Kong, D.; Wang, X. Rolling Bearing Fault Diagnosis Based on VMD-CNN-SLSTM Model. In Proceedings of the 2025 IEEE International Conference on Mechatronics and Automation (ICMA), Beijing, China, 3–6 August 2025; pp. 78–84. [CrossRef]
35. Andalib, N.; Selimi, M. Effective Quantization Technique for Enhancing Model Performance on Resource-Constrained Devices. In Proceedings of the 2025 14th Mediterranean Conference on Embedded Computing (MECO), Budva, Montenegro, 10–14 June 2025; pp. 1–4. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.